REDROB HACKATHON

5-Day Execution Battle Plan

Hour-by-hour tasks, deliverables, priorities, and submission checklist

Overview — How to Read This Plan

This is your hour-by-hour guide across 5 days. Each day has a clear primary focus. Tasks are prioritized: CRITICAL tasks must be done before bed. HIGH tasks should be done that day. MEDIUM tasks can slip by 24 hours without destroying your submission. LOW tasks are polish only if time allows.

Priority Level	What It Means	What Happens If You Skip It
CRITICAL	The submission physically cannot be made without this	Submission fails, gets disqualified, or scores 0
HIGH	Directly impacts your NDCG@10 score (50% of total)	Measurable score drop — likely to lose top placements
MEDIUM	Improves score quality but system works without it	Moderate score reduction — still submittable
LOW	Polish, UI presentation, nice-to-haves	Zero impact on ranking score — skip if pressed for time

> **GOLDEN RULE:** Backend logic is your score. Frontend is your proof-of-work. A perfect backend with no frontend = Stage 1 rejection. A perfect frontend with broken backend = Stage 2 failure. Do backend first, always.

Day	Theme	End-of-Day Goal
Day 1	Foundation & Data	Project structure set up, dataset fully understood, config.py and data_loader.py working, JD parsed
Day 2	Core Logic — Backend	Honeypot detector, skill scorer, career analyzer all working on sample_candidates.json
Day 3	Scoring Engine & Full Pipeline	rank.py runs end-to-end on sample_candidates.json, producing valid CSV output. Pre-computation done.
Day 4	Full 100K Run + Frontend	rank.py produces valid submission.csv from full candidates.jsonl. Frontend skeleton running with Results Table.
Day 5	Polish, Validate & Submit	All 3 submission components ready. Sandbox hosted. CSV validated. GitHub clean. Submitted before deadline.

DAY 1 Foundation & Data — Set up everything. Understand the data. Build the skeleton.

Day 1 is about zero surprises on Day 2. When Day 1 ends, you should be able to load any candidate from the dataset and print every field. You should have read every file in the dataset. You should have a running config.py that is the single source of truth for all your constants.

Time Block	Task	Deliverable / Done When	Priority
Morning (0–3h)	Set up GitHub repo. Create project structure: all files as empty modules. Create requirements.txt. Push initial commit.	GitHub repo exists. All .py files exist. pip install works.	CRITICAL
Morning (3–5h)	Read all dataset files carefully: job_description.docx, redrob_signals_doc.docx, submission_spec.docx, candidate_schema.json, sample_submission.csv, validate_submission.py. Take notes.	You can answer: What are the 6 scoring components? What makes a honeypot? What is the exact CSV format?	CRITICAL
Afternoon (5–7h)	Build config.py. Add: hard skills list with synonyms and weights, preferred skills, consulting firm list, location map, experience curve values, behavioral thresholds, disqualifier penalty values.	config.py importable. All constants defined. No hardcoded values in any other file.	CRITICAL
Afternoon (7–9h)	Build jd_parser.py. Parse job_description.docx. Extract hard skills, preferred skills, disqualifiers, implicit requirements. Store as requirements dict. Compute JD embedding using sentence-transformers.	python jd_parser.py prints the full requirements dict. JD embedding vector exists.	CRITICAL
Evening (9–11h)	Build data_loader.py. Stream candidates.jsonl line by line. Parse each line. Yield candidate dict. Test on sample_candidates.json first, then on a 1000-line slice of candidates.jsonl.	data_loader.py loads all 50 sample candidates without errors. All fields accessible.	CRITICAL
Night (11h+)	Run precompute_embeddings.py skeleton. Start embedding all 100K candidates using sentence-transformers all-MiniLM-L6-v2. This takes 20-40 min — start it and let it run overnight.	precompute_embeddings.py running in terminal. Will produce candidate_embeddings.npy by morning.	HIGH

DAY 1 END STATE CHECK: You can run: python data_loader.py and see 50 candidates print. You can run: python jd_parser.py and see parsed requirements. precompute_embeddings.py is running in the background.

DAY 2 Core Logic — All scoring modules implemented and tested on sample data.

Day 2 is the most important day. The four modules you build today determine 85% of your final score. Build them carefully. Test each one on sample_candidates.json before moving to the next. Do not skip tests.

Time Block	Task	Deliverable / Done When	Priority
Early Morning (0–1h)	Check that precompute_embeddings.py completed overnight. Verify candidate_embeddings.npy exists and has 100K rows. If still running, let it continue and work on other modules first.	candidate_embeddings.npy: shape (100000, 384). No NaN values.	CRITICAL
Morning (1–4h)	Build honeypot_detector.py. Implement all 4 check categories: timeline consistency, skill proficiency validity, education timeline, overall data consistency. Return honeypot_confidence 0-1 per candidate.	On sample_candidates.json: at least 1-2 candidates should flag as suspicious. A Marketing Manager claiming expert ML skills should flag. Print honeypot results for all 50 sample candidates.	CRITICAL
Morning (4–7h)	Build skill_scorer.py. Implement: skill normalisation, category matching, trust formula (proficiency × duration × endorsements × assessment), semantic blend (70% keyword + 30% cosine similarity). Test on 5 candidates manually.	ML Engineer with FAISS/NLP experience scores 0.7+. Marketing Manager with same skills in profile but no matching career descriptions scores <0.2.	CRITICAL
Afternoon (7–10h)	Build career_analyzer.py. Implement: consulting firm classification, product company detection, production signal keyword scan of descriptions, career trajectory analysis, industry relevance scoring. Compute career_quality_score.	TCS-only candidate scores career_quality near 0.1. Product company ML engineer scores 0.8+. Run on all 50 sample candidates and verify results make intuitive sense.	CRITICAL
Evening (10–12h)	Build behavioral_scorer.py. Implement: availability_multiplier (open_to_work × last_active × response_rate), notice_period_modifier, platform_quality_score (GitHub, interview completion, profile completeness). Combine into behavioral_score.	Inactive candidate (180 days, response rate 0.03) gets multiplier 0.20. Active candidate (7 days, response rate 0.72) gets multiplier 1.10.	HIGH
Night (12h+)	Build experience_scorer.py and location_scorer.py. Both are simple lookup tables from config.py. Quick to implement — 30 min each.	YoE=7 scores 1.0. YoE=2 scores 0.3. Pune scores 1.0. Outside India with no relocation scores 0.15.	HIGH

DAY 2 END STATE CHECK: You have 6 working scorer modules. Each tested individually on sample_candidates.json. Print a table of scores for all 50 sample candidates from each module. Verify the top candidates per module make intuitive sense — they should be ML/AI engineers with product company experience.

DAY 3 Pipeline & Reasoning — Full end-to-end pipeline working. Valid CSV output. Reasoning generated.

Day 3 is integration day. Connect all your Day 2 modules into a single pipeline. By end of Day 3, python rank.py must produce a valid submission.csv on sample_candidates.json. This is the milestone that unlocks Day 4.

Time Block	Task	Deliverable / Done When	Priority
Morning (0–3h)	Build scoring_engine.py. Combine all 6 component scores using the composite formula: final_score = (0.35×skill + 0.30×career + 0.10×experience + 0.10×location + 0.05×education) × behavioral_multiplier × disqualifier_penalty. Normalise to 0-1.	scoring_engine.py takes a candidate dict + features dict → returns float 0-1. Test on 10 manually selected candidates from sample. Scores should be intuitive.	CRITICAL
Morning (3–5h)	Build ranker.py. Process all candidates in sample_candidates.json. Compute score per candidate. Sort descending. Handle ties (candidate_id ascending). Select top N (all if <100 candidates).	python ranker.py on 50 sample candidates produces a sorted list. No duplicate ranks. Scores non-increasing.	CRITICAL
Afternoon (5–8h)	Build reasoning_generator.py. For each top-ranked candidate generate a 1-2 sentence reasoning. Must reference actual profile data: title, YoE, specific company/skill from career_history, location, notice period, availability indicator. No generic text.	Every reasoning text is unique. Each mentions real data from that specific candidate. No candidate has the same reasoning as any other. Print all reasonings and manually check 5 of them.	CRITICAL
Afternoon (8–9h)	Build output_writer.py. Write top candidates to CSV: candidate_id, rank, score, reasoning. UTF-8 encoding. Exactly the sample_submission.csv format.	output_writer.py produces a CSV that matches the sample_submission.csv format exactly.	CRITICAL
Afternoon (9–10h)	Build rank.py as main entrypoint. Wire all modules together. Accept --candidates and --out arguments. Run full pipeline from load to CSV write.	python rank.py --candidates ./sample_candidates.json --out ./test_submission.csv runs without errors. Produces valid CSV.	CRITICAL
Evening (10–11h)	Run validate_submission.py on your test_submission.csv. Fix every error it reports.	'Submission is valid.' — zero errors.	CRITICAL
Evening (11–12h)	Manual quality review: print the top 10 candidates from test_submission.csv. Verify they are ML/AI engineers at product companies. Verify no Marketing Managers in top 10. Verify reasonings are specific.	Top 10 pass your manual sense-check. No obvious keyword stuffers or honeypots in top ranks.	HIGH
Night (12h+)	If embedding precomputation is done: run precompute_features.py to pre-cache all non-embedding features for 100K candidates. This speeds up Day 4 full run.	candidate_features.pkl exists. Reduces Day 4 full run time.	MEDIUM

DAY 3 END STATE CHECK: python rank.py --candidates ./sample_candidates.json --out ./test.csv runs in under 30 seconds, produces a validate_submission.py-passing CSV, with specific non-hallucinated reasoning per candidate. This is your core submission working.

DAY 4 Full Run + Frontend — 100K ranking complete. Frontend skeleton live. Sandbox running.

Day 4 has two parallel tracks: full 100K ranking run (backend) and frontend build. Start the full ranking run first thing in the morning — it will take 30-90 minutes depending on your hardware. While it runs, build the frontend.

Time Block	Task	Deliverable / Done When	Priority
Early Morning (0–1h)	Run rank.py on the FULL candidates.jsonl. Time it. Monitor memory usage. Fix any out-of-memory or timeout issues.	rank.py completes on full 100K dataset. Produces submission.csv with exactly 100 rows.	CRITICAL
Morning (1–2h)	Run validate_submission.py on the 100K submission.csv. Fix all errors.	'Submission is valid.' on 100K output. This is your actual submission file.	CRITICAL
Morning (2–3h)	Manual review of top 20 candidates in the 100K submission. Are they the right people? Do rankings make sense? Spot-check 5 reasonings for accuracy.	You are satisfied the top 10 are real ML/AI engineers. No glaring wrong answers in top 20.	HIGH
Morning (3–5h)	Set up FastAPI backend server (api.py). Implement /api/status, /api/jd, /api/rank, /api/validate, /api/export endpoints as defined in the Frontend Spec document.	FastAPI server runs locally. /api/status returns {status:'ready'}. /api/jd returns parsed JD. /api/rank on 10 candidates returns scored results with features.	CRITICAL
Afternoon (5–8h)	Build frontend: App Shell (S1), Home Dashboard (S2), Results Table (S7) with all columns. These three screens are the minimum viable sandbox.	Frontend runs locally. Shows ranked results table. Judges can see top candidates with scores and reasoning.	CRITICAL
Afternoon (8–10h)	Build S6 (Processing Tracker) and S4 (Candidate Input with sample pre-loader). Connect frontend to backend /api/rank. End-to-end: click 'Load Sample' → click 'Run Ranking' → see results in table.	Full demo loop works: load → rank → view results. No errors.	CRITICAL
Evening (10–11h)	Build S8 (Candidate Detail modal) with Score Breakdown chart, Skills Table, Behavioral Grid. This is the transparency screen judges care about most.	Clicking any row in S7 opens S8 with full score breakdown. Score breakdown matches the API response features.	HIGH
Evening (11–12h)	Build S3 (JD Analyzer) and S10 (Export Panel with CSV download). These complete the judge evaluation flow.	S3 shows parsed JD requirements. S10 downloads valid CSV.	HIGH
Night (12h+)	Deploy sandbox to HuggingFace Spaces, Streamlit Cloud, or Vercel. Test that it works on a deployed URL, not just localhost.	Sandbox accessible at a public URL. Demo runs end-to-end on that URL.	CRITICAL

DAY 4 END STATE CHECK: You have a valid submission.csv from 100K candidates. You have a working frontend at a public URL. The demo loop (load sample → run ranking → view results → download CSV) works end-to-end without errors.

DAY 5 Polish, Validate & Submit — Everything reviewed, validated, and submitted before deadline.

Day 5 is not a build day. It is a validation, polish, and submission day. Stop adding features. Focus entirely on making what you have clean, correct, and submittable.

Time Block	Task	Deliverable / Done When	Priority
Morning (0–2h)	Full end-to-end smoke test: fresh clone of your GitHub repo → pip install → precompute (if needed) → python rank.py → validate_submission.py. Verify it works from zero on a clean machine.	Fresh clone produces a valid CSV. No missing dependency errors.	CRITICAL
Morning (2–3h)	Review top 10 candidates in submission.csv one final time. Manually read each reasoning. Check for: correct title mentioned, correct company mentioned, correct skills mentioned, no hallucinated facts.	All 10 reasonings are specific, accurate, non-hallucinated. You would bet your score on these 10 being correct.	CRITICAL
Morning (3–4h)	Fill out submission_metadata.yaml completely. All fields: team name, approach summary, model used, runtime, hardware spec, sandbox URL, GitHub URL, framework used. Both declaration checkboxes = true.	submission_metadata.yaml complete. No empty fields.	CRITICAL
Morning (4–5h)	Write README.md. Must include: one-command run instruction, dependency install, pre-computation step, expected output, system requirements (CPU, RAM), sandbox URL, approach in 3 paragraphs.	README.md complete. Another person could run your system following only the README.	CRITICAL
Afternoon (5–7h)	Final sandbox test: open sandbox at public URL. Run the full demo: load sample candidates → run ranking → inspect top candidate detail → download CSV → verify CSV format. Fix any broken UI.	Sandbox demo works end-to-end at public URL. All screens navigate correctly.	CRITICAL
Afternoon (7–8h)	Build S9 (Analytics Dashboard) if not done. At minimum: score distribution histogram, top-10 spotlight, disqualifier impact table.	S9 shows meaningful charts. Not blank.	MEDIUM
Afternoon (8–9h)	GitHub repo cleanup: remove debug prints, remove test files, remove large cached files (add to .gitignore), verify the repo is clean and professional.	GitHub repo has clean commit history. No debug output. No secrets or API keys committed.	HIGH
Evening (9–10h)	Run validate_submission.py one final time on your submission.csv. Screenshot the 'Submission is valid.' output.	validate_submission.py passes. Screenshot saved.	CRITICAL
Evening (10–11h)	Submit: upload submission.csv to competition portal, submit GitHub link, submit sandbox URL, upload PDF deck if required.	All 3 (or 4 with deck) submission components uploaded to competition portal. Confirmation email received.	CRITICAL
Evening	BUFFER: fix any last-minute submission	Submission confirmed on	CRITICAL

Time Block	Task	Deliverable / Done When	Priority
(11–12h)	portal issues. Resubmit if needed. (3 submissions max — use carefully.)	competition portal before deadline.	

FINAL SUBMISSION CHECKLIST: CSV file (validated) | GitHub repo (clean, working, README complete) | Sandbox (public URL, demo runs) | submission_metadata.yaml (filled) | PDF deck if required. ALL FIVE before deadline.

Time Budget Summary

Approximate time allocation across all 5 days:

Area	Hours Budgeted	% of Total	Why This Much
Core Scoring Logic (honeypot, skill, career, behavioral)	16 hours	27%	Determines 85% of your score. No shortcuts.
Pipeline Integration (scoring engine, ranker, rank.py)	8 hours	13%	Connecting working modules. Should be fast if modules are clean.
Reasoning Generator	5 hours	8%	Stage 4 evaluates this directly. Must be specific and non-hallucinated.
Pre-computation (embeddings, features)	4 hours	7%	Run once. Critical for meeting 5-minute constraint.
Frontend (all 10 screens)	14 hours	23%	Minimum viable sandbox: 8h. Full spec: 14h.
Testing & Validation	6 hours	10%	Manual reviews of top 10, validate_submission.py, fresh-clone test.
README, metadata, GitHub cleanup	3 hours	5%	Judges dock points for incomplete submission metadata.
Buffer / bug fixing	4 hours	7%	Always needed. Build it in.

IF YOU RUN OUT OF TIME: Priority order for cutting scope: (1) Cut S5 config screen — default weights are fine. (2) Cut S9 analytics — judges rarely need it. (3) Simplify S8 to just score breakdown + reasoning (skip skills table). (4) NEVER cut S7 (results table), S8 (score breakdown), S10 (CSV export), or S6 (processing tracker) — these are mandatory for sandbox evaluation.

Risk Register — What Can Go Wrong

Risk	Likelihood	Impact	Mitigation
precompute_embeddings.py runs out of time or memory	Medium	HIGH — no semantic matching	Use MiniLM-L6-v2 (smallest fast model). Process in batches of 1000. Start on Day 1 night.
rank.py takes >5 minutes on full 100K	Medium	CRITICAL — disqualification	Pre-compute ALL features offline. Online step should only load cached files + sort.
validate_submission.py fails on your CSV	Low	CRITICAL	Run it after EVERY change to output_writer.py. Do not submit without it passing.
Sandbox deployment fails (HuggingFace, Streamlit)	Medium	HIGH — Stage 1 rejection	Test deployment on Day 4. Have a fallback: Colab notebook that runs the demo is acceptable.
Top 10 contains keyword stuffers	Medium	HIGH — NDCG@10 destroyed	After every pipeline change, manually print top 10 titles. If you see non-ML titles, your skill trust formula has a bug.
Honeypot rate >10% in top 100	Low	CRITICAL — disqualification	Print honeypot_confidence for all top-100 candidates. Any >0.7 must be excluded.
Reasoning is generic/templated	Medium	HIGH — Stage 4 penalty	Print all 10 top reasonings side by side. If they look copy-pasted with different names, rewrite the template.
GitHub repo has missing files or broken install	Low	CRITICAL	Test fresh clone on Day 5. Non-negotiable.

Recommended Tech Stack

Component	Recommended Tool	Why
Embedding model	sentence-transformers (all-MiniLM-L6-v2)	Fast, CPU-compatible, 384 dims, no API key needed, free
Candidate streaming	ijson or simple json.loads per line	Avoids loading 465MB into RAM at once
Feature caching	numpy (npy) for embeddings, pickle for feature dicts	Fast load, standard, no database needed
Scoring engine	Pure Python + numpy for vector operations	No external ML framework needed — scoring is arithmetic
Backend API	FastAPI + Uvicorn	Minimal setup, async, auto-generates API docs judges can inspect
Frontend framework	React (Vite) or Next.js	Fast to scaffold, component-based, easy charts with Recharts or Chart.js
Charts	Recharts (React) or Chart.js	Both work well for score histograms and bar charts
Sandbox hosting	HuggingFace Spaces (Gradio/Streamlit) or Vercel (Next.js)	Free, fast deployment, judges can access without login
CSV validation	validate_submission.py (provided)	Run this — do not write your own validator
Word document reading	python-docx	Read job_description.docx in jd_parser.py

---

END OF 5-DAY EXECUTION PLAN

Day 1: Foundation | Day 2: Logic | Day 3: Pipeline | Day 4: Full Run + Frontend | Day 5: Submit